

Analyse gas

Mesures réalisées avec `forge snapshot` (Foundry, optimiseur activé, 200 runs, `via_ir` désactivé) et `forge build --sizes`. Le script `tools/gas_experiments.py` automatise les variantes : il applique une modification temporaire au contrat, mesure, puis restaure le fichier d'origine.

Q1 : gas de transfer()

51 348 gas par appel (`gas-report.txt`, moyenne sur 2 appels Hardhat).

Le snapshot Foundry indique 78 944 gas pour `test_Transfer`, mais cette valeur englobe le harnais de test. Le chiffre de 51 348 correspond à une transaction réelle et constitue la mesure pertinente.

Décomposition. Le coût de base d'une transaction est de 21 000 gas. La fonction lit le solde source avec un `SLOAD`, puis effectue deux `SSTORE` : le débit de la source modifie un slot déjà occupé, soit 2 900 gas, tandis que le crédit du destinataire passe un slot de zéro à une valeur non nulle, soit 20 000 gas. L'événement `Transfer` produit un `LOG3`, soit environ 1 500 gas. Le reste couvre le décodage des arguments et l'exécution du bytecode.

Le poste dominant est le `SSTORE` du destinataire. Un second transfert vers la même adresse coûterait nettement moins, le slot étant alors déjà non nul.

Q2 : avec et sans unchecked

Variante	Gas	Écart
Bloc <code>unchecked</code>	78 944	référence
Arithmétique vérifiée	79 142	+198

Le compilateur insère un contrôle de dépassement autour de chaque opération arithmétique depuis Solidity 0.8.0. Deux opérations sont concernées dans `_transfer`, soit environ 99 gas par contrôle.

Le bloc `unchecked` est justifié mathématiquement : la soustraction est protégée par le contrôle `bal < amount` qui précède immédiatement, et l'addition ne peut pas déborder puisque la somme des soldes reste invariablement égale à `totalSupply`, elle-même bornée par `uint256`.

Q3 : erreur custom contre require avec chaîne

Variante	Taille du bytecode	Gas <code>transfer()</code> nominal
<code>revert</code> <code>InsufficientBalance(bal,</code> <code>amount)</code>	2 347 octets	78 944
<code>require(bal >= amount,</code> <code>'MyToken: insufficient</code> <code>balance')</code>	2 421 octets	78 944

Le chemin nominal est identique, la mesure donne exactement 0 d'écart, puisque le contrôle de solde réussit et qu'aucune donnée de `revert` n'est produite. La différence porte donc sur deux autres axes.

Taille du bytecode : +74 octets pour la version `require`. La chaîne de caractères est stockée littéralement dans le bytecode, ce qui augmente le coût de déploiement à raison de 200 gas par octet. L'erreur custom n'occupe qu'un sélecteur de 4 octets calculé par hachage de sa signature.

Données de revert. L'erreur custom encode `InsufficientBalance(uint256,uint256)` en 4 octets de sélecteur suivis des deux arguments ABI-encodés, soit 68 octets, et transmet au client les valeurs `available` et `required` exploitables programmatiquement. La version `require` produit `Error(string)` contenant la chaîne, sans aucune donnée structurée.

Le coût du chemin de revert mesuré sur `test_RevertIf_InsufficientBalance` est de **37 770 gas** pour l'erreur custom. La variante `require` n'est pas mesurable sur ce test : celui-ci attend le sélecteur de l'erreur custom et échoue par construction, ce qui illustre que les deux mécanismes ne sont pas interchangeables du point de vue des consommateurs du contrat.

Q4 : gas du constructeur

662 059 gas au déploiement (`gas-report.txt`, section Deployments), soit 1,1 % de la limite de gas par bloc.

Décomposition approximative.

Le stockage du bytecode est le poste dominant. La facturation est de 200 gas par octet de code déployé. Le contrat compilé par Hardhat pèse 2 200 octets, soit **440 000 gas**, environ 66 % du total.

L'écriture de `name` et `symbol` occupe deux slots de storage. Chaque passage d'un slot nul à une valeur non nulle coûte 20 000 gas, soit **40 000 gas**.

Les écritures du constructeur portent sur `owner`, `totalSupply` et `balanceOf[msg.sender]`, soit trois SSTORE depuis zéro et **60 000 gas**. Le champ `decimals` est déclaré `immutable` : sa valeur est inscrite dans le bytecode et n'occupe aucun slot, ce qui économise 20 000 gas.

Le coût de base de la transaction est de 21 000 gas.

Ces postes totalisent environ 561 000 gas. L'écart avec la mesure de 662 059 correspond à l'exécution du code de création, aux données d'appel contenant les arguments du constructeur et à l'expansion mémoire.

Le poste le plus coûteux reste le stockage du bytecode. La réduction de la taille du code a donc un effet plus marqué sur le déploiement que sur les appels.

Q5 : influence du paramètre `optimizer runs`

<code>optimizer_runs</code>	Gas transfer()	Écart
1	79 414	+470
200	78 944	référence
1 000	78 881	-63

Le paramètre exprime le nombre d'appels attendus sur la durée de vie du contrat et arbitre entre taille du bytecode et coût d'exécution. Une valeur de 1 privilégie un bytecode compact, donc un déploiement bon marché, au prix d'appels plus coûteux. Une valeur élevée déroule davantage de code et inline plus agressivement, réduisant le coût par appel.

La courbe s'aplatit rapidement : passer de 1 à 200 économise 470 gas par transfert, alors que passer de 200 à 1 000 n'en économise que 63 supplémentaires. Sur un contrat de cette taille, les opportunités d'optimisation sont épuisées bien avant 1 000 runs. La valeur 200 retenue dans le TP constitue un compromis raisonnable pour un token destiné à un usage fréquent.